In addition to searching for books written by Joshua with a rating no less than 4.7, we can add another condition to check whether the books match a tag (say, `tag = "Software"`). If it's a match, we expect the score to increase; otherwise, it has no effect on the result. For this, we use the `should` clause, discussed in the next section.

2.5.4 The should clause

The `should` clause behaves like an `OR` operator. That is, if the search words are matched against the `should` query, the relevancy score is bumped up. If the words do not match, the query does not fail; the clause is ignored. The `should` clause is more about increasing the relevancy score than affecting the results.

The next listing adds the `should` clause to the `bool` query. It tries to match the search text if documents have a `Software` tag.

Listing 2.22 Using a `should` query to increase relevancy

```
GET books/_search
{
  "query": {
    "bool": {
      "must": [{"match": {"author": "Joshua"}}],
      "must_not":[{"range":{"amazon_rating":{"lt":4.7}}}],
      "should": [{"match": {"tags": "Software"}}]
    }
  }
}
```

should clause with a match query

This query returns the result (omitted here) with an increased score: 2.267993, compared to an earlier score of 1.9459882 (you can run the queries to observe the scores).

If you are curious, change the query to include a non-matching word (`tags` equals `"Recipes"`, for example) and retry the query. The query will not fail, but the score will be the same, which proves that the `should` query affects only the scores.

Continuing with the flow, the final clause is `filter`, which works exactly like the `must` clause but doesn't affect the score. Let's see it in action in the next section.

2.5.5 The filter clause

Let's improve our query a bit more, this time by filtering out books published before 2015 (that is, we don't want any books to appear in our resultset that were published before 2015). We use a `filter` clause for this purpose, and any results that don't match the filter criteria are dropped. The following query adds the `filter` clause with a `release_date` clause.

Listing 2.23 A `filter` clause, which doesn't affect relevancy

```
GET books/_search
{
  "query": {
```